

## LECTURE NOTES

# Algorithm and Programming

## Format Input dan Output

**Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M**



## Learning Outcomes

1. LO1: Apply syntax and functions in the C language and AI problem-solving sessions related to the topics.

## Outline Materi

1. Output
2. Input
3. Problem-Solving Session with AI: Validation of Basic Syntax (Input & Output)

# Output

File berisi karakter dan angka terkait. Karena semua karakter dan angka diwakili dalam bit di komputer, bahasa C memperlakukan file sebagai serangkaian byte. (8 bit make up 1 byte.) Serangkaian byte juga disebut aliran. Bahkan, bahasa C memperlakukan semua aliran file secara setara, meskipun beberapa aliran file mungkin berasal dari disk atau tape drive, dari terminal, atau bahkan dari printer.

Selain itu, di C, ada tiga aliran file yang telah dibuka sebelumnya untuk Anda:

- Stdin - Input standar untuk membaca.
- stdout - output standar untuk menulis.
- Stderr - Kesalahan standar untuk menulis pesan kesalahan.

Biasanya, tautan aliran file input (stdin) standar ke keyboard Anda, sedangkan aliran file output standar (stdout) dan standar kesalahan standar (stderr) menunjuk ke layar terminal Anda. Juga, banyak sistem operasi memungkinkan Anda untuk mengarahkan kembali aliran file -file ini.

Bahkan, Anda sudah menggunakan stdout. Ketika Anda mengeksekusi fungsi `printf ()` dalam pelajaran terakhir, Anda sebenarnya mengirim output ke aliran file default, stdout, yang menunjuk ke layar Anda.

Input dan output dilakukan dengan urutan byte yang disebut stream:

- Dalam operasi input, byte mengalir ke memori utama dari perangkat, seperti keyboard, drive solid-state, koneksi jaringan, dan sebagainya.
- Dalam operasi output, byte mengalir dari memori utama ke perangkat, seperti layar komputer, printer, drive solid-state, koneksi jaringan, dan sebagainya.

Ketika eksekusi program dimulai, program memiliki akses ke tiga aliran:

- Aliran input standar, yang terhubung ke keyboard,
- Aliran output standar, yang terhubung ke layar, dan
- Aliran error standar, yang juga terhubung ke layar.

Fungsi output input dalam pemrograman C masuk ke dalam dua kategori, yaitu fungsi output input (I/O) yang diformat dan fungsi output input (I/O) yang tidak diformat.

- Fungsi input-output yang diformat menerima atau menyajikan data dalam format tertentu. Perpustakaan standar terdiri dari berbagai fungsi yang melakukan operasi output dan input.

- Fungsi input-output yang tidak diformat tidak dapat mengontrol format membaca dan menulis data. Fungsi-fungsi ini adalah bentuk paling dasar dari input dan output dan mereka tidak memungkinkan untuk menyediakan input atau menampilkan output dalam format yang diinginkan pengguna, itulah sebabnya kami menyebutnya fungsi input-output yang tidak diformat. Fungsi input-output yang tidak diformat diklasifikasikan ke dalam dua kategori sebagai fungsi input-output karakter dan fungsi input-output string.

| No | Formatted                                                                                                                | Unformatted                                                                                                                                                                                   |
|----|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Fungsi I/O yang diformat memungkinkan untuk memasok input atau menampilkan output dalam format yang diinginkan pengguna. | Fungsi I/O yang tidak diformat adalah bentuk paling dasar dari input dan output dan mereka tidak memungkinkan untuk memasok input atau tampilan output dalam format yang diinginkan pengguna. |
| 2  | printf () dan scanf () adalah contoh untuk fungsi input dan output yang diformat.                                        | getch (), getche (), getChar (), gets (), puts (), putchar () dll adalah contoh fungsi output input yang tidak diformat.                                                                      |
| 3  | Fungsi input dan output yang diformat berisi spesifikasi format dalam sintaksnya.                                        | Fungsi input dan output yang tidak diformat tidak mengandung spesifikasi format dalam sintaksnya.                                                                                             |
| 4  | Fungsi I/O yang diformat digunakan untuk menyimpan data lebih ramah pengguna.                                            | Fungsi I/O yang tidak diformat digunakan untuk menyimpan data lebih rumit.                                                                                                                    |
| 5  | Fungsi I/O yang diformat digunakan dengan semua tipe data.                                                               | Fungsi I/O yang tidak diformat digunakan terutama untuk tipe data karakter dan string.                                                                                                        |
| 6  | Contoh I/O yang diformat:<br><pre>#include&lt;stdio.h&gt; #include&lt;conio.h&gt;  void main() {   int a;</pre>          | Contoh I/O yang tidak diformat:<br><pre>#include&lt;stdio.h&gt; #include&lt;conio.h&gt;  void main() {   char ch ;</pre>                                                                      |

|                                                                                                                                              |                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>clrscr(); printf("Enter value of a:"); scanf("%d", &amp;a); printf(" a = %d", a); getch(); }</pre> <pre>Enter value of a:5↵ a = 5</pre> | <pre>clrscr(); printf("Press any character:"); ch = getche(); printf("\n You pressed :") putchar(ch); getch(); }</pre> <pre>Press any character: L You pressed: L</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Output

Instruksi keluaran (*output*) digunakan untuk menampilkan hasil proses, pada layar monitor, dicetak pada *printer*, atau ditulis ke *file*. Berikut adalah data-data operasi *output* pada bahasa C.

### A. printf();

printf () adalah fungsi output yang diformat yang digunakan untuk menampilkan beberapa informasi pada unit output standar. Ini didefinisikan dalam file header standar *stdio.h*. Jadi, untuk menggunakan printf (), kita harus menyertakan file header *stdio.h* dalam program.

### B. putchar();

Fungsi Puth () digunakan untuk mencetak karakter ke layar di lokasi kursor saat ini. Ini adalah fungsi output karakter yang tidak diformat dan didefinisikan dalam file header *conio.h*.

### C. puts();

Puts () adalah fungsi output string yang tidak diformat dan dapat menulis atau mencetak string ke layar. Didefinisikan dalam file header standar *stdio.h*.

Fungsi ini juga dapat menambahkan karakter dan memindahkan kursor ke baris baru.

## Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

### Pertanyaan Diskusi:

Dalam bahasa C, semua file, keyboard, dan layar dianggap sebagai *stream* byte. Menurut Anda, apa keuntungan dari konsep *stream* ini dalam pengolahan input dan output di berbagai perangkat?

#### Hint:

- Bahas: stream = aliran data byte universal → memudahkan I/O ke device berbeda (keyboard, file, printer).
- Diskusi: programmer tidak perlu peduli asal data (disk, keyboard, network).
- Contoh: printf mengirim ke stdout (layar), fprintf ke file.

### Pertanyaan Diskusi:

Jelaskan perbedaan utama antara *formatted I/O* dan *unformatted I/O* dalam C! Berikan situasi praktis di mana programmer sebaiknya menggunakan *unformatted I/O*.

#### Hint:

- Bahas:
  - Formatted: printf, scanf → dengan format string.
  - Unformatted: getch(), putchar() → langsung input/output karakter atau string tanpa format.
- Contoh penggunaan: input password (getch() tidak echo ke layar), debugging karakter per karakter.

### Pertanyaan Diskusi:

Fungsi output seperti printf(), putchar(), dan puts() punya kegunaan berbeda. Diskusikan perbedaan fungsi-fungsi ini dan kapan sebaiknya programmer memilih salah satunya dalam program?

#### Hint:

- printf(): output terformat (angka, string, variabel kompleks).
- putchar(): cetak 1 karakter ke layar, posisi kursor saat ini.
- puts(): cetak string + pindah baris otomatis.
- Contoh: puts() cocok untuk cetak kalimat cepat; printf() untuk format laporan.

### Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C Variables and Print Output: [Youtube](#)).

### Belajar Mandiri



Pelajari secara mandiri terkait subtopik ini pada:

- Kochan, S. G. (2015). *Programming in C*. Pearson education. Chapter 16, Page 347.

# Input

Operasi Input: Fungsi/Pengoperasian Mendapatkan Data ke Memori Menggunakan Perangkat I/O Standar (Keyboard, Disk, dll.)

Fungsi pustaka standar yang terkait dengan operasi input adalah:

A. `scanf()`;

`SCANF ()` adalah fungsi input yang diformat yang digunakan untuk membaca data yang diformat dari perangkat input standar dan didefinisikan dalam *stdio.h*.

Pemformatan input yang tepat dapat dilakukan dengan `scanf`. Setiap pernyataan `SCANF` berisi string kontrol format yang menjelaskan format data yang akan dimasukkan. String kontrol format terdiri dari penentu konversi dan karakter literal. Fungsi `SCANF` memiliki kemampuan pemformatan input berikut:

- Memasukkan semua jenis data.
- Memasukkan karakter tertentu dari aliran input.
- Melewati karakter tertentu dalam aliran input.

B. `getchar()`;

`getChar ()` juga merupakan fungsi input karakter. Itu membaca karakter dan menerima input sampai carriage return dimasukkan (mis. Sampai masuk ditekan). Itu didefinisikan dalam file header *stdio.h*.

C. `getch()`;

`getch ()` adalah fungsi input karakter. Ini adalah fungsi input yang tidak diformat yang berarti tidak memungkinkan pengguna untuk membaca input dalam formatnya. Ini membaca karakter dari keyboard tetapi tidak menggemakan karakter yang ditekan dan mengembalikan karakter yang ditekan. Itu didefinisikan dalam file header *conio.h*.

D. `getche()`;

Seperti `getch ()`, `getche ()` juga merupakan fungsi input karakter. Ini adalah fungsi input yang tidak diformat yang berarti tidak memungkinkan pengguna untuk membaca input dalam formatnya. Perbedaan antara `getch ()` dan `getche ()` adalah bahwa `getche ()` menggemakan karakter yang ditekan. `getche ()` juga mengembalikan karakter yang ditekan seperti `getch ()`. Ini juga didefinisikan dalam file header *conio.h*.

E. `gets()`;

`gets ()` adalah fungsi input string. Itu membaca string dari keyboard. Membaca string sampai masuk ditekan. Itu didefinisikan dalam file header *stdio.h*.

## Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

### Pertanyaan Diskusi:

Apa perbedaan utama antara `scanf()` dengan fungsi input karakter seperti `getchar()`, `getch()`, `getche()`, dan `gets()`? Diskusikan contoh kasus di mana `scanf()` lebih cocok digunakan dibandingkan fungsi input karakter!

#### Hint:

- Bahas `scanf()` membaca berbagai tipe data dengan format tertentu (angka, float, char, string).
- Fungsi `getchar()`, `getch()`, `getche()` → fokus input karakter tunggal, `gets()` → string.
- Contoh: masukkan umur dan nama → `scanf()` cocok. Masukkan password (karakter per karakter tanpa echo) → `getch()` lebih aman.

### Pertanyaan Diskusi:

Fungsi `gets()` dulu sering digunakan untuk input string, tetapi sekarang sering dianggap tidak aman. Diskusikan risiko penggunaan `gets()` dan apa alternatif modern yang lebih aman dalam bahasa C?

#### Hint:

- Bahas: `gets()` tidak membatasi panjang input → bisa menyebabkan buffer overflow.
- Konsekuensi: program crash, celah keamanan.
- Alternatif: `fgets()` dengan batas panjang input → lebih aman.

### Pertanyaan Diskusi:

Apa perbedaan antara `getch()` dan `getche()` dalam hal echo karakter ke layar? Kapan programmer lebih memilih menggunakan `getch()` daripada `getche()`?

#### Hint:

- `getch()`: input karakter tanpa echo → cocok untuk input password atau sandi.
- `getche()`: input karakter dengan echo → cocok saat ingin menampilkan karakter yang ditekan.
- Diskusikan situasi praktis: buat login sederhana, user tidak ingin password terlihat → pakai `getch()`.



### Belajar Mandiri

Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini ([Get User Input in C Programming: Youtube](#)).



### Belajar Mandiri

Pelajari secara mandiri terkait Pengenalan AI pada:

- Kochan, S. G. (2015). *Programming in C*. Pearson education. Chapter 16, Page 347.

## 2.3

## Sesi Pemecahan Masalah dengan AI: Validasi Sintaks Dasar (Input & Output)

**HAICOLA Section:** Mahasiswa dapat menggunakan AI Code Assistant (seperti **Gemini** atau **ChatGPT**) untuk melakukan pengecekan sintaks, memahami error, dan memperbaiki kode C secara efisien.

Pemrograman C dimulai dengan menguasai fungsi dasar untuk berinteraksi dengan pengguna: **Output** (`printf`) dan **Input** (`scanf`). Kesalahan dalam bagian ini seringkali merupakan kesalahan sintaks dasar yang dapat dengan mudah diidentifikasi dan diperbaiki dengan bantuan AI Code Assistant (seperti Gemini).

### Fokus: Sintaks `printf` dan `scanf`

Meskipun terlihat sederhana, kesalahan dalam *formatting* dan pengelolaan alamat memori adalah masalah umum yang harus dikuasai mahasiswa untuk memenuhi **LO 1**. Tantangan utama meliputi:

1. **Format Spesifikasi (*Format Specifiers*):** Kesalahan penggunaan *specifier* (misalnya, menggunakan `%d` untuk `float`, atau `%f` untuk `int`).
2. **Sintaks `scanf` dan Alamat Memori:** Kelupaan menggunakan operator alamat (`&`) di depan variabel saat menggunakan `scanf`, yang dapat menyebabkan *crash* program atau nilai yang tidak terduga.

## Contoh Praktis: Menggunakan AI untuk Validasi Sintaks

Anda dapat menggunakan AI Code Assistant untuk memverifikasi sintaks Anda, **mempercepat proses *debugging***, dan fokus pada pemecahan masalah yang lebih kompleks.

### Skenario 1: Kesalahan *Format Specifier* di `printf`

Mahasiswa mencoba mencetak nilai *floating-point* tetapi menggunakan spesifikasi *integer*.

#### Kode yang Bermasalah:

```
#include <stdio.h>

int main() {
    float pi = 3.14159;
    // Kesalahan: Menggunakan %d (integer) untuk float
    printf("Nilai PI adalah: %d\n", pi);
    return 0;
}
```

#### Permintaan (Prompt) ke AI:

*"Kode C saya mencetak nilai PI dengan output yang salah (misalnya, '0' atau angka acak). Ini kodanya: [masukkan kode di atas]. Apa yang salah dengan cara saya mencetak variabel 'pi'?"*

#### Respon AI yang Diharapkan:

*AI akan menjelaskan bahwa variabel `pi` adalah `float` tetapi format specifier yang digunakan adalah `%d` (untuk `integer`). AI akan menyarankan perbaikan dengan mengubah `%d` menjadi `%f` atau `%.2f` (untuk dua desimal) dan menjelaskan pentingnya mencocokkan tipe data dengan format specifier.*

### Skenario 2: Kesalahan Alamat Memori di `scanf`

Mahasiswa mencoba membaca input dari pengguna tetapi lupa menggunakan operator alamat.

#### Kode yang Bermasalah:

```
#include <stdio.h>

int main() {
    int umur;
    printf("Masukkan Umur Anda: ");
    // Kesalahan: Lupa menggunakan operator alamat (&)
    scanf("%d", umur);

    printf("Umur Anda: %d\n", umur);
    return 0;
}
```

#### Permintaan (Prompt) ke AI:

"Program C saya crash atau tidak menyimpan input dari user ke variabel 'umur'. Ini kodenya: [masukkan kode di atas]. Mengapa 'umur' tidak menerima input?"

### Respon AI yang Diharapkan:

AI akan menunjukkan bahwa `scanf` memerlukan **alamat memori** dari variabel tempat data akan disimpan. Anda harus menggunakan operator alamat (&) di depan `umur`, yaitu `scanf("%d", &umur)`; . AI akan menekankan bahwa penggunaan & adalah sintaks vital untuk fungsi input di C.

### Praktik Terbaik untuk Mahasiswa

- **Sintaks adalah Kunci:** Gunakan AI untuk memverifikasi sintaks dasar Anda ketika Anda mendapatkan *error* yang tidak jelas. Ini adalah cara tercepat untuk menguasai **LO 1: Menerapkan sintaks**.
- **Pahami &:** Selalu pastikan Anda memahami kapan harus menggunakan operator alamat (&) untuk Input (`scanf`) dan kapan tidak. Minta AI menjelaskan perbedaan *syntax* ini.
- **Latihan Employability Skill:** Mendapatkan umpan balik instan tentang kesalahan sintaks dasar melalui AI membantu mengembangkan kemampuan **Self-Management** dan **Attention to Detail**, memungkinkan Anda belajar secara mandiri dari kesalahan pemrograman umum.

**Post-Test**

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

**1. Fungsi printf() dalam bahasa C termasuk ke dalam kategori fungsi ...**

- A. Output tidak diformat
- B. Input berformat
- C. Output berformat
- D. Input tidak diformat

**2. Fungsi scanf() digunakan untuk ...**

- A. Mencetak string ke layar
- B. Membaca input dengan format tertentu
- C. Membaca karakter tanpa echo
- D. Menulis output ke file

**3. Dalam bahasa C, stdin secara default terhubung ke perangkat ...**

- A. Layar monitor
- B. Keyboard
- C. Printer
- D. Harddisk

**4. Fungsi manakah yang termasuk fungsi input/output tidak diformat?**

- A. printf()
- B. scanf()
- C. putchar()
- D. fprintf()

**5. Perbedaan getch() dan getche() terletak pada ...**

- A. Kemampuan membaca string
- B. Apakah karakter yang ditekan di-echo ke layar
- C. Format input yang diatur
- D. Lokasi output di file

# Kesimpulan

- Output
  - Instruksi keluaran (output) digunakan untuk menampilkan hasil proses, pada layar monitor, dicetak pada printer, atau ditulis ke file.  
Contoh output pada bahasa C:
    - `printf();`
    - `putchar();`
    - `putch();`
    - `puts();`
- Input
  - Instruksi masukan (input) adalah fungsi yang digunakan untuk membaca data. Contoh input pada bahasa C:
    - `scanf();`
    - `getchar();`
    - `getch()`
    - `getche()`
    - `gets();`

## Daftar Pustaka

- Paul Deitel & Harvey Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN: 978-0-13-739839-3. Chapter 9
- Reading from and Writing to Standard I/O: <http://aelinik.free.fr/c/ch05.htm>
- [https://www.codesansar.com/c-programming/putch.htm#:~:text=The%20putch\(\)%20function%20is,h%20](https://www.codesansar.com/c-programming/putch.htm#:~:text=The%20putch()%20function%20is,h%20)
- [https://www.cs.utah.edu/~germain/PPS/Topics/C\\_Language/file\\_IO.html](https://www.cs.utah.edu/~germain/PPS/Topics/C_Language/file_IO.html)
- <https://www.youtube.com/watch?v=-LqUMHoBo6o>
- [https://www.youtube.com/watch?v=01QsW5\\_fuek](https://www.youtube.com/watch?v=01QsW5_fuek)
- Kochan, S. G. (2015). Programming in C. Pearson education. Chapter 16, Page 347.

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban:** C

**Alasan:**

printf() digunakan untuk menampilkan output dengan format tertentu → contoh fungsi output berformat.

2. **Jawaban:** B

**Alasan:**

scanf() adalah fungsi input berformat: membaca angka, string, char sesuai format yang ditentukan.

3. **Jawaban:** B

**Alasan:**

stdin = input standar → default-nya terhubung ke keyboard untuk membaca input user.

4. **Jawaban:** C

**Alasan:**

putch() adalah fungsi output karakter tidak diformat → mencetak karakter di posisi kursor.

5. **Jawaban:** B

**Alasan:**

getch() membaca karakter tanpa echo. getche() membaca karakter sambil menampilkan (echo) ke layar.